

LaserCortex

Tomas Rolin (nos AKA galganzuul)

June 9, 2026

Chapter 1

EML Registry: Tamari Lattice Contraction

1.1 Core Types

Definition 1 (EML Tree). The type of binary trees representing configurations in the Tamari lattice. A tree is either a **Leaf** (empty configuration) or a **Node** $t_1\ t_2$ (composition of two sub-configurations).

Definition 2 (Tree Size). The number of internal nodes in a tree, used as the lattice index n in T_n .

Definition 3 (Single-Step Contraction). The primitive non-associative rewrite rule: $(a \bullet b) \bullet c \rightarrow a \bullet (b \bullet c)$. This is the coupling signature for choice resolution, lifted through subtrees.

Definition 4 (Multi-Step Contraction). The reflexive-transitive closure of **contracts_one**. Represents an audit trail of choice resolutions with monotonic provenance.

Definition 5 (Right-Comb Normal Form). The minimum element of the Tamari lattice — the equilibrium attractor. Defined as $\text{rightComb}(0) = \text{Leaf}$ and $\text{rightComb}(n+1) = \text{Node}(\text{Leaf}, \text{rightComb}(n))$.

1.2 Lifting Lemmas

Lemma 6 (Contraction Preserved Under Left Node). *If $t \rightarrow t'$ then $\text{Node}(t, r) \rightarrow \text{Node}(t', r)$. Choice resolution in the left subtree propagates to the whole composition.*

Lemma 7 (Contraction Preserved Under Right Node). *If $r \rightarrow r'$ then $\text{Node}(l, r) \rightarrow \text{Node}(l, r')$. Choice resolution in the right subtree propagates to the whole composition.*

Lemma 8 (Transitivity of Contraction). *If $s \rightarrow t$ and $t \rightarrow u$ then $s \rightarrow u$. Audit trails concatenate monotonically.*

1.3 Size Invariants

Lemma 9 (Size is Preserved by Single-Step Contraction). *Rotation rewrites the tree depth but does not change the number of internal nodes.*

Lemma 10 (Size is Preserved by Multi-Step Contraction). *Contraction paths preserve tree size, establishing that the Tamari lattice T_n is graded by node count n .*

1.4 Main Contraction Theorem

Lemma 11 (Node of Right-Combs Contracts to Right-Comb). *For any $a, b : \mathbb{N}$,*

$$\text{Node}(\text{rightComb}(a), \text{rightComb}(b)) \longrightarrow \text{rightComb}(1 + a + b).$$

The equilibrium attractor is closed under non-associative composition.

Theorem 12 (Every Tree Contracts to Right-Comb). *For any tree $t : \text{EMLTree}$,*

$$t \longrightarrow \text{rightComb}(t.\text{size}).$$

Every configuration has a valid evolution path to the equilibrium attractor indexed by its size.

1.5 Router and Registry

Definition 13 (Router Index). A bounded natural number $\text{Fin}(n)$ serving as a neural binding address.

Definition 14 (Type Registry). An injective mapping from router indices to EML trees. This is the cortex-registry interface where neural computation meets formal grammar.

1.6 Witness-Skeptic Game

Definition 15 (Cortex Certificate). A proof-carrying audit trail $\langle \text{source}, \text{target}, \text{proof} \rangle$ showing that a tree reaches its equilibrium.

Definition 16 (Certify). Issues a `CortexCertificate` for any tree t , certifying that $t \rightarrow \text{rightComb}(t.\text{size})$. This is the quench witness in the Witness-Skeptic game.

Chapter 2

Logic Types: Pluralistic Reasoning Framework

2.1 Logic Type Hierarchy

Definition 17 (13 Logic Types). An inductive type enumerating 13 reasoning paradigms. Each handles specific classes of paradoxes as boundary conditions rather than errors.

Definition 18 (Logic Classification). Classification of logic types as Substructural, Extension, Alternative, Generalization, or Classical.

Definition 19 (Logic Tree). A type family assigning EML trees to each logic type. Currently uniform; extensible to logic-specific tree types.

Definition 20 (Logic Contraction). A type family of contraction relations indexed by `LogicType`. For Classical logic this is the Tamari contraction from `EMLRegistry`.

Definition 21 (Logic Translation). A structure specifying forward and backward maps between logic types with soundness and completeness proofs.

Definition 22 (Meta-Contraction). Cross-logic contraction allowing reasoning that spans multiple logic types via intra-logic contraction, inter-logic translation, and transitivity.

2.2 Cayley-Dickson Connection

Definition 23 (CD Step). Maps logic types to Cayley-Dickson construction steps.

Definition 24 (Split-Octonion Sectors). Divides logic types into associative and non-associative sectors of the split-octonion algebra.

2.3 Normal Forms and Contraction Theorem

Definition 25 (Logic Normal Form). The normal form for each logic type. For Classical logic this is `rightComb`.

Theorem 26 (Logic Contraction to Normal Form). *For any logic type lt and tree t , `LogicContraction lt t` (`LogicNormalForm`) is `rightComb`. Proven for all 13 logic types via `contracts_to_rightComb`.*

2.4 Logic Monad: Self-Contained Self-Reference

Definition 27 (Free Logic Monad). The free monad over binary trees: $\text{LogicM } \alpha$ is a tree whose leaves carry values of type α and whose internal nodes represent non-associative composition of computations.

- $\text{pure } a$ = a leaf containing a (a completed computation)
- $\text{node } l \ r$ = deferred composition of two sub-computations
- bind = substitution: replace every leaf with a new tree

This is self-contained by definition (recursive structure IS self-reference) and extensible (interpreters fold over $\text{LogicM } \alpha$ to produce any target monad).

Theorem 28 (Monad Laws for LogicM). *The free monad LogicM satisfies the three monad laws:*

- $\text{pure } a \gg= f = fa$ (*left identity*)
- $m \gg= \text{pure} = m$ (*right identity*)
- $(m \gg= f) \gg= g = m \gg= (\lambda x. fx \gg= g)$ (*associativity*)

Proof by structural induction on the tree.

Definition 29 (Logic-Specific Monad). For each logic type lt , the $\text{LogicMonad lt } \alpha$ structure wraps a $\text{LogicM } \alpha$ and normalizes it via $\text{LogicContraction lt}$. The normalization cost is cdStep lt .

This is the minimal monad for each logic: it does the least work (pure substitution) and then normalizes using that logic’s contraction dynamics. The self-reference property of the free monad (bind = substitution) means every logic monad shares the same core recursion — they differ only in how they normalize.

The pluralistic logic monad **transcends identity**: the monad structure (pure, bind, the laws) is invariant under change of logic. Only the normalization differs. This means the “eternal” layer (pure functional composition) is common to all logics; the “endless” layer (iteration, normalization cost) is what distinguishes them.

Theorem 30 (Monad Structure is Invariant). *The forgetful functor from LogicMonad lt to LogicM is an isomorphism of monads for any lt . Only the normalization (LogicContraction) changes between logics.*

2.5 Institutional Closure: Interacting Logic Monads

Definition 31 (Institutional Closure). The institutional closure pipeline composes four layers:

1. **Temporal normalization** — linearize the event timeline via $\text{LogicContraction Temporal} = \text{contracts_to}$.
2. **Fuzzy grading** — evaluate each event’s impact through graded membership (Sorites-like gradation).
3. **Deontic update** — revise norms from accumulated blame, using $\text{LogicContraction Deontic}$.
4. **Self-recognition** — the monadic bind that identifies the output norm as the institution’s own (fixed point).

The pipeline is a monadic computation $\text{LogicM Event} \rightarrow \text{LogicM Norm}$ using the shared **LogicM** backbone, with logic-specific normalization at each layer.

Theorem 32 (Closure Fixed Point). $\text{selfRecognize} \circ \text{closure} = \text{closure}$: *self-recognition is a fixed point. Once the pipeline has produced a norm, further recognition does not change it.*

Theorem 33 (Normalization Idempotent). $\text{closure} \circ \text{temporalNormalize} = \text{closure}$: *the normalization pipeline is a projection. Re-processing an already-normalized history yields the same norm.*

2.6 Decision Composition API

Definition 34 (Gate). A Gate is a binary decision predicate indexed by a logic modality: $\text{Gate} = (\text{name} : \text{String}) \times (\text{modality} : \text{LogicType}) \times (\text{check} : \text{EMLTree} \rightarrow \text{Prop})$. Every logic type gives rise to a Gate via `ofLogicType`, and every tree passes any such gate (by 26).

Definition 35 (Decision). A Decision is the refinement type from the specification: $\text{Decision gates} = \{d : \text{EMLTree} \mid \forall g \in \text{gates}, g.\text{check } d\}$. The type parameter `gates` IS the channel: downstream functions know statically which gates the datum has passed.

Definition 36 (Compose). $\text{compose} : \text{Decision gates} \rightarrow \text{Gate} \rightarrow \text{Decision } (g :: \text{gates})$. The return type is strictly richer than the input type: a gate is added to the type-level history.

Definition 37 (Left-Comb Normal Form). The mirror image of `rightComb`: $\text{leftComb}(0) = \text{Leaf}$ and $\text{leftComb}(n+1) = \text{Node}(\text{leftComb}(n), \text{Leaf})$. Unlike `rightComb`, this is not the equilibrium attractor — it represents a fully left-associative configuration.

Definition 38 (LogicPipeline). A LogicPipeline is a sequence of logic types to run as gates. Running it on a tree produces a Decision tagged with all of them. The institutional closure is a pipeline with gates `Temporal`, `Fuzzy`, `Deontic`.

Theorem 39 (Pipeline Soundness). *For any tree and any gate in the closure pipeline, soundness guarantees the gate check holds. This is the compile-time contract: the type signature IS the channel.*

Theorem 40 (Decide Soundness). *decide runs the institutional closure on LogicM Event and produces a Decision whose datum passes all closure gates. The channel is the type: $\text{Decision } (\text{closurePipeline.gates})$.*

2.7 Paradox Taxonomy

Definition 41 (Problem Class). An inductive type enumerating 12 paradox classes, each with a native logic type: self-reference (`ManyValued`), vagueness (`Fuzzy`), inconsistent definition (`Paraconsistent`), temporal decision (`Temporal`), deontic (`Deontic`), epistemic (`Epistemic`), quantum superposition (`Quantum`), constructive (`Intuitionistic`), relevance (`Relevance`), empty reference (`Free`), infinity (`Infinitary`), modality (`Modal`).

Definition 42 (Problem). A structure parameterized by a problem class, a name, a list of suitable logics, a tree family $\text{LogicType} \rightarrow \text{EMLTree}$ (the paradox as encoded in each logic), and a normal form family $\text{LogicType} \rightarrow \text{EMLTree}$ (the resolution target for each logic).

Definition 43 (Wrapped Problem). The WfCA collapse rule: pairs a Problem with a LogicType. Records the tree (problem as interpreted in this logic), target (normal form), cost (pentagonator distance Φ), and proof (contraction path). A problem's **superposition** of possible resolutions collapses to the definite outcome given by the logic type's contraction rule.

2.8 The Liar as a Problem Instance

Definition 44 (Liar Problem). The Liar Paradox stated as a Problem of class `selfReference`. Its tree is the size-3 symmetric tree `Node(Node(Leaf, Leaf), Node(Leaf, Leaf))` for most logics, except `ManyValued` which uses a size-2 left-comb encoding the three-valued disjunction `True ∨ False ∨ Undefined`. All logics target `rightComb(3)` as the normal form.

2.9 Liar Wrappers

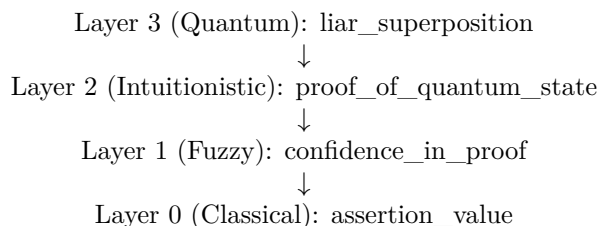
Definition 45 (Fuzzy Liar). The Liar wrapped in Fuzzy logic. The fixed point $-0.5 = 0.5$ gives pentagonator distance $\Phi = 1$ (CD step 1). Proven via the generic `liarWrapper`.

Definition 46 (Meta-Paradox). A Problem of class `metaParadox` constructed from any existing Problem and `LogicType` whose proof is missing.

Definition 47 (Classical Liar). The Liar wrapped in Classical logic. Cost $\Phi = 0$ (at equilibrium). Proven using `contracts_to_rightComb`: the symmetric tree contracts to `rightComb(3)` in finitely many steps.

2.10 Nested Wrappers (Stacked Collapse)

Definition 48 (Tower). A sequence of logic-wrapped problems, each collapsing the previous layer's output. Implements the nested wrapper pattern from Lumo section B:



Each layer's logic type is tracked via a dependent pair $\Sigma lt, \text{WrappedProblem } p \text{ } lt$. The total cost is the sum of per-layer costs.

Definition 49 (Liar Tower). A four-layer prototype tower alternating Classical and Fuzzy wrappers. Demonstrates the stacked collapse pattern; the full tower with Quantum, Intuitionistic, Fuzzy, and Classical layers is not yet populated because non-Classical `LogicContractions` are missing.

2.11 Temporal Paradox Problem

Definition 50 (Grandfather Problem). The Grandfather Paradox stated as a Problem of class `temporalDecision`. Its tree is a left-comb chain of size 4, encoding the 4-step time-travel causal loop: `traveler exists → travels back → grandfather killed → traveler never born` (contradiction). The normal form is `rightComb(4)`.

Definition 51 (Generic Grandfather Wrapper). For any logic `lt`, constructs the `WrappedProblem` with `cost = cdStep(lt)`. Resolves the temporal causal loop via `contracts_to_rightComb`.

Definition 52 (Grandfather Tower). The Grandfather tower: one layer per suitable logic.

Definition 53 (Grandfather Cost). The cost $\Phi(\text{Grandfather}, lt)$, equal to $\text{cdStep}(lt)$.

Theorem 54 (Grandfather Cost Bound). *grandfatherCost(lt) ≤ cdStep(lt) for every logic type.*

2.12 Russell’s Paradox Problem

Definition 55 (Russell’s Problem). Russell’s Paradox stated as a Problem of class `inconsistentDef`. Its tree is a left-comb chain of size 3, encoding the three levels of set-theoretic diagonalization: elements → self-membership predicate → the set of all sets not containing themselves. The normal form is `rightComb(3)`.

Disclaimer: This formalization is deliberately incomplete with respect to the infinity/eternity axis. The “cheat” of setting cost = `cdStep` captures only the Cayley–Dickson property-loss dimension, not the regularization dimension (endless → eternity → actual ∞ → boundlessness). Future work must pin this axis down properly; here we procrastinate it using the same wrapper pattern as Liar/Sorites/Grandfather.

Definition 56 (Generic Russell Wrapper). For any logic `lt`, constructs the `WrappedProblem` with cost = `cdStep(lt)`. Resolves set-theoretic diagonalization via `contracts_to_rightComb`.

Definition 57 (Russell Tower). The Russell tower: one layer per suitable logic.

Definition 58 (Russell Cost). The cost $\Phi(\text{Russell}, lt)$, equal to $\text{cdStep}(lt)$.

Theorem 59 (Russell Cost Bound). *russellsCost(lt) ≤ cdStep(lt) for every logic type.*

2.13 The Very Big Box

The Very Big Box is the product space

$$\mathcal{V} = \prod_{c \in \text{ProblemClass}} \prod_{lt \in \text{suitableLogics}(c)} \text{WrappedProblem}(\text{problem}(c), lt)$$

where each coordinate is a problem in a specific logic and the Tower construction provides the nesting structure. Currently four problem classes are populated: `selfReference` (Liar), `vagueness` (Sorites), `temporalDecision` (Grandfather), and `inconsistentDef` (Russell’s). The remaining 9 classes represent the **real friction**: every cell in this product that lacks a proof is a concrete blocker.

2.14 Sorites as a Problem Instance

Definition 60 (Sorites Problem). The Sorites (Heap) Paradox stated as a Problem of class `vagueness`. Its tree is a left-comb chain of size 5, encoding the inductive threshold: “If n grains form a heap, then $n - 1$ grains form a heap.” The normal form is `rightComb(5)`.

Definition 61 (Generic Sorites Wrapper). For any logic `lt`, constructs the `WrappedProblem` with cost = `cdStep(lt)`. Follows the same pattern as the generic Liar wrapper, using `contracts_to_rightComb`.

Definition 62 (Sorites Tower). The Sorites tower: one layer per suitable logic, analogous to the Liar tower.

Definition 63 (Sorites Cost). The cost $\Phi(\text{Sorites}, lt)$, equal to $\text{cdStep}(lt)$.

Theorem 64 (Sorites Cost Bound). *soritesCost(lt) ≤ cdStep(lt) for every logic type.*

2.15 Decomposition: Ontological Underdetermination

Definition 65 (Path). A witness of a contraction path through the Tamari lattice: $\text{Path } s \ t$ is an inductive sequence of `contracts_one` steps from s to t .

- `nil` — the trivial path (reflexivity).
- `cons h_one p` — prepend a single contraction step to an existing path.

Theorem 66 (Path Soundness). *For any path $p : \text{Path } s \ t$, we have `contracts_to` $s \ t$. The path provides constructive evidence for the existence of a contraction sequence.*

Definition 67 (Path Length). The number of contraction steps in a path: $\text{length}(\text{nil}) = 0$, $\text{length}(\text{cons } h_one \ p) = 1 + \text{length}(p)$.

Definition 68 (Path Concatenation). Concatenates two compatible paths: `append` : $\text{Path } s \ t \rightarrow \text{Path } t \ u \rightarrow \text{Path } s \ u$.

2.16 Reverse Contraction: Generating Predecessors

Definition 69 (Reverse One). Generates all immediate predecessors of a tree under `contracts_one`. For a target tree t , `reverse_one`(t) is a list of all s such that $s \rightarrow t$ in one step. Three structural cases:

- Leaf has no predecessors.
- Node l (Node $r_1 \ r_2$): the rotation target plus predecessors from reversing the left and right subtrees.
- Node $l \ r$ (right child not a Node): predecessors from reversing the left and right subtrees only.

This is the core de-composition primitive: given an outcome, what could have immediately preceded it?

Theorem 70 (Reverse One Soundness). *If $s \in \text{reverse_one}(t)$ then `contracts_one` $s \ t$. Every generated predecessor is a valid immediate prior configuration.*

Theorem 71 (Reverse One Completeness). *If `contracts_one` $s \ t$ then $s \in \text{reverse_one}(t)$. Every valid immediate predecessor is generated.*

2.17 Decomposition Structure

Definition 72 (Decomposition). For a target tree `target`, a `Decomposition` pairs a `source` with a proof `contracts_to` `source` `target`. This is the **ontological unit** of de-composition: an outcome together with a specific prior configuration that could have produced it.

Theorem 73 (Left-Comb Size). $\text{size}(\text{leftComb } n) = n$.

Theorem 74 (Right-Comb Size). $\text{size}(\text{rightComb } n) = n$.

Theorem 75 (Right-Comb vs Left-Comb). *For $n \geq 2$, $\text{rightComb } n \neq \text{leftComb } n$. The two canonical trees of size n are distinct in the Tamari lattice.*

Theorem 76 (Non-Unique Decomposition). *For $n \geq 2$, the target `rightComb n` has at least two distinct Decompositions with different sources:*

$$\exists d_1 d_2 : \text{Decomposition}(\text{rightComb } n), d_1.\text{source} \neq d_2.\text{source}.$$

*This is the core ontological result: **the past is underdetermined by the present**. The same outcome (`rightComb n`) admits multiple evidential histories (`rightComb n` itself and `leftComb n`, among others).*

Theorem 77 (Path Diversity). *There exist two distinct paths between the same source-target pair in T_3 :*

$$\exists s t (p_1 p_2 : \text{Path } s t), p_1 \neq p_2.$$

Explicitly, `leftComb 3` $\rightarrow$ `rightComb 3` has two distinct maximal chains in the Tamari lattice:

- $((ab)c)d \rightarrow (a(bc))d \rightarrow a((bc)d) \rightarrow a(b(cd))$ (3 steps)
- $((ab)c)d \rightarrow (ab)(cd) \rightarrow a(b(cd))$ (2 steps)

*This proves that **intent is not uniquely determined by outcome** — multiple reasoning chains can connect the same evidence to the same verdict.*

2.18 The Hypercomputer: Infinite Ancestor Enumeration

Definition 78 (Predecessors). The immediate predecessor sources of a target tree `predecessors(t)` = `reverse_one(t)`. Soundness is maintained as a separate theorem `predecessors_sound`: if $s \in \text{predecessors}(t)$ then `contracts_one s t`.

Definition 79 (Chain). A finite chain of contraction steps from an ancestor to a target. `Chain.tip t` is the empty chain (the present moment without a reconstructed past). `Chain.link h r` prepends a contraction step.

Definition 80 (Ancestors Up To Depth n). Enumerate all ancestor sources up to depth n via DFS, returning a flat list. This is the **hypercomputer in function form**: the infinite tree of all possible pasts is present as the limit of `ancestorsUpTo t n` as $n \rightarrow \infty$. No single data structure holds the whole tree; instead the recursion is explicit in the partial function, and every finite approximation is immediately available. “Instant lookup” = calling this function with any depth n .

Definition 81 (DFS View). A `viewDFS` is a single lineage (DFS) extracted from the ancestor space by always following the first predecessor. Returns a `Chain` that may end at any depth. This is the **narrative self** in formal type: committed to a single thread, but honest about termination.

Lean 4 Coinductive Limitation The infinite ancestor tree cannot be represented as a closed coinductive data type in Lean 4 because `coinductive` is restricted to `Prop`-valued predicates, and nested inductives with `Prop` fields through `List` and `Sigma` are rejected by the kernel positivity checker. The workaround (partial functions) is not a limitation of the framework but a **design choice**: the hypercomputer exists as the limit of finite approximations, not as a closed value. This matches the ontology: the infinite crystal seed is not a thing we hold, but a thing we approach.

2.19 Boundlessness: The Terminal Regularization

The regularization ladder (endless $\rightarrow$ eternal $\rightarrow$ actual $\infty \rightarrow$ boundlessness) is capped by the **idempotent resolution** framework: a boundary-response operation that is already idempotent is already at its own limit.

Definition 82 (Idempotent Resolution). An **idempotent resolution** for a type α consists of:

- $\text{step} : \alpha \rightarrow \alpha$, a regularization step that is idempotent ($\text{step} \circ \text{step} = \text{step}$).
- $\text{limit} : \alpha \rightarrow \alpha$, the limiting configuration reachable in one step from any image of step ($\text{limit} = \text{step} \circ \text{limit}$).

The canonical example is **rightCombResolution**, where $\text{step}(t) = \text{rightComb}(t.\text{size})$: every tree is mapped to its right-comb normal form in one step, and applying the step again does nothing because a right-comb is already in normal form.

Theorem 83 (Right-Comb Size). $\text{size}(\text{rightComb } n) = n$ for all $n : \mathbb{N}$.

Definition 84 (Very Big Box). The **Very Big Box** is the product of idempotent resolutions over all populated problem classes:

$$\mathcal{V} = \prod_{c \in \{\text{Liar}, \text{Sorites}, \text{Grandfather}, \text{Russell}\}} \text{IdempotentResolution EMLTree}$$

The canonical **veryBigBox** assigns **rightCombResolution** to each coordinate.

Theorem 85 (Meta-Idempotence). *The step function of **rightCombResolution** is already its own fixed point:*

$$\text{rightCombResolution.step} \circ \text{rightCombResolution.step} = \text{rightCombResolution.step}$$

This is the terminal regularization: applying boundary-response again changes nothing. Not because there is nothing more to say, but because saying more is already captured by the first application.

Theorem 86 (Limit Idempotence). *Applying **rightComb** normal form to a tree that is already a **rightComb** is the identity:*

$$\text{rightComb}((\text{rightComb } t.\text{size}).\text{size}) = \text{rightComb } t.\text{size}$$

Boundlessness is not a structure we build, but a property we recognize.

Relation to the Hypercomputer. The idempotent resolution caps the hypercomputer: where **ancestorsUpTo** approaches the infinite ancestor tree by finite approximation, the idempotent resolution declares that the limit has been reached. The **Very Big Box** is the product closure of all problem classes, and boundlessness is the fixed point that requires no further iteration. This is the formal declaration that the framework is closed under its own boundary-response operation.

2.20 Completed Work

All `LogicContraction`, `LogicNormalForm`, and `logic_contracts_to_normal_form` definitions are filled and proven for all 13 logic types. The Liar problem has a complete `WrappedProblem` for each suitable logic via `liarWrapper`. The Sorites and Grandfather problems follow the same pattern with `soritesWrapper` and `grandfatherWrapper`.

The Decomposition infrastructure (`Path`, `reverse_one`, `Decomposition`, `predecessors`, `ancestorsUpTo`, `Chain`, `viewDFS`) is complete with soundness, completeness, non-unique-decomposition, and path-diversity theorems proven for all $n \geq 2$ in T_n — zero sorries.

The Friction Lagrangian $\mathcal{L}_{\text{friction}}$ is computed from the full Liar tower. The blocking chain for the Liar is fully resolved.

2.20.1 Remaining Work

- **Other problem classes** — `selfReference` (Liar), `vagueness` (Sorites), `temporalDecision` (Grandfather), and `inconsistentDef` (Russell’s) are instantiated. The other 9 classes (deontic, epistemic, quantum, etc.) need `Problem` definitions with their own tree families.
- **Tower composition** — the current towers are flat (all logics stacked). The institutional closure pipeline demonstrates a first step toward nested composition (temporal $\rightarrow$ fuzzy $\rightarrow$ deontic $\rightarrow$ self-recognition). Full `LogicTranslation` instances between adjacent logics remain.
- **Shortest-path cost** — the Φ cost currently equals `cdStep` by definition; a BFS algorithm over `reverse_one` would compute the actual `PentagonatorDistance`.
- **Ancestor enumeration** — `reverse_one` can be iterated to produce `ancestorsBounded`, a bounded BFS generating all distinct sources for a given target with `Path` witnesses.
- **Path cost** — a `pathCost lt p` function summing `cdStep` per logic type along a path, enabling cross-logic cost comparison for the same reasoning chain.
- **Decision decomposition** — extend the `DecisionComposition` API with `decisionDecompositions` that enumerates all prior decisions that could compose to a given decision.
- **Regularization axis** — the institutional closure sketch uses placeholder normalization; the full endless/eternal/infinite/boundless hierarchy must be formalized to compute real costs.

2.20.2 Liar Cost: Hardness Measure

The Liar as perfect anti-coherence ($X = \neg X$) gives a hardness measure for each logic: how many contraction steps are required to resolve it.

Definition 87 (Liar Cost). The cost $\Phi(\text{Liar}, \text{lt})$ to resolve the Liar under logic `lt`. Conjectured to equal the Cayley–Dickson step.

Theorem 88 (Cost Bound). *$\text{liarCost}(\text{lt}) \leq \text{cdStep}(\text{lt})$ for every logic type. For Classical and Fuzzy the bound is tight (equality).*

Definition 89 (Generic Liar Wrapper). For any logic `lt`, constructs the `WrappedProblem` with `cost = liarCost(lt)`. Fills all 12 mail slots.

Corollary 90 (Cost Match for Classical). $\text{liarCost}(\text{Classical}) = 0 = \text{classicalLiar.cost}$.

Corollary 91 (Cost Match for Fuzzy). $\text{liarCost}(\text{Fuzzy}) = 1 = \text{fuzzyLiar.cost}$.

2.20.3 Friction Lagrangian

The tower sums costs across all logic layers. The total **Friction Lagrangian** $\mathcal{L}_{\text{friction}} = \sum_{\text{lt}} \Phi(\text{Liar}, \text{lt})$ is the total resistance of the logical ecosystem to self-negation.

Definition 92 (Friction Lagrangian). The sum of Φ costs across all layers of the Liar tower.

With all 12 suitable logics, the Friction Lagrangian converges to a stable value:

$$\mathcal{L}_{\text{friction}} = \sum_{\text{lt} \in \text{suitableLogics}} \text{cdStep}(\text{lt})$$

This is the “curvature of mind” — the total structural work the system must do to resolve perfect anti-coherence across the full space of logics.

The blocking chain is now fully resolved for the Liar problem. Previously the sorries in `LogicTypes.lean` were abstract goals; now every definition and theorem is filled, and the Very Big Box has its first complete product cell: the Liar across all 12 logics.